

Competitive Programming

Section 44 Editorial: String Algorithms II

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Trie Prefix Count	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
2.7	Tests that matter	5
3	B. Palindrome Queries	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
3.7	Tests that matter	7
4	C. Distinct Substrings	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
4.7	Tests that matter	9
5	What to carry forward	10

1 Editorial overview

1.1 Detailed solution guide

A. Trie Prefix Count. Each trie node represents exactly one prefix: the string on the path from the root to that node. During insertion, increment `cnt` at the root and after moving to each next node. Therefore, after all insertions, `cnt[v]` is the number of dictionary words whose path passes through `v`, which is precisely the number of words with that prefix. A query walks the prefix characters; a missing edge means zero matches.

B. Palindrome Queries. For the local constraints, comparing mirrored characters is the clearest exact solution. For substring `[l,r]`, compare `s[l]` with `s[r]`, then `s[l+1]` with `s[r-1]`, and so on. If all pairs match, the substring equals its reverse. If one pair differs, the two strings differ at a mirrored position, so it is not a palindrome. In a larger contest version, the same condition can be checked with forward and reverse rolling hashes.

C. Distinct Substrings. The local solution enumerates every substring and inserts it into a set. A set stores one representative of each equal substring, so its final size is exactly the number of distinct substrings. The optimized red-level version is a suffix automaton: each state contributes `len[v] - len[link[v]]` new substrings, because those are exactly the lengths whose first end-position class is represented by `v`.

Common pitfalls. Trie counters count prefixes, not terminal words only. Palindrome queries must convert input indices consistently. For distinct substrings, enumeration is pedagogical and exact at small limits, but a suffix automaton is required for large external tasks.

This section shows three string representations.

- A uses a trie. Each inserted word increments counters on the nodes along its path, so the counter at the prefix endpoint is exactly the number of words with that prefix.
- B checks palindromes by comparing mirrored characters. For larger limits this can be replaced by forward/reverse hashes, but the local proof is exact.
- C counts distinct substrings. The simplest local implementation may enumerate substrings at small limits; the red-level invariant is the suffix automaton contribution `len[v] - len[link[v]]` per state.

The editorial code favors transparent correctness where limits allow it, while the lesson explains the optimized structures used in larger external tasks.

2 A. Trie Prefix Count

2.1 Restatement

Given dictionary words and prefix queries, count words having each prefix.

2.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

2.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

2.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Node{array<int,26> nx{}; int cnt=0;};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,q;if(!(cin>>n>>q)) return 0; v
```

2.6 Full Python solution

```

import sys
def main():
    lines=sys.stdin.read().splitlines(); n,q=map(int,lines[0].split()); nxt=[]; cnt=[]
    nxt.append({}); cnt.append(0)
    for w in lines[1:1+n]:
        v=0; cnt[v]+=1
        for ch in w:
            if ch not in nxt[v]: nxt[v][ch]=len(nxt); nxt.append({}); cnt.append(0)
            v=nxt[v][ch]; cnt[v]+=1
    out=[]
    for p in lines[1+n:1+n+q]:
        v=0; ok=True
        for ch in p:
            if ch not in nxt[v]: ok=False; break
            v=nxt[v][ch]
        out.append(str(cnt[v] if ok else 0))
    print("\n".join(out))
if __name__=="__main__": main()

```

2.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

3 B. Palindrome Queries

3.1 Restatement

For each substring query, print whether the substring is a palindrome.

3.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

3.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. \square

3.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(c
```

3.6 Full Python solution

```
import sys
def main():
    s=sys.stdin.readline().strip(); q=int(sys.stdin.readline()); out=[]
    for _ in range(q):
        l,r=map(int,sys.stdin.readline().split()); sub=s[l-1:r]; out.append("YES" if sub==su
    print("\n".join(out))
if __name__=="__main__": main()
```

3.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

4 C. Distinct Substrings

4.1 Restatement

Print the number of distinct non-empty substrings of a string.

4.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

4.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

4.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct St{map<char,int> nx; int link=-1,len=0;};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(cin>>s)) return 0; vec
```

4.6 Full Python solution

```

import sys
def main():
    s=sys.stdin.readline().strip(); next=[]; link=[]; length=[]
    next.append({}); link.append(-1); length.append(0); last=0
    for ch in s:
        cur=len(next); next.append({}); length.append(length[last]+1); link.append(0); p=last
        while p!=-1 and ch not in next[p]: next[p][ch]=cur; p=link[p]
        if p==-1: link[cur]=0
        else:
            q=next[p][ch]
            if length[p]+1==length[q]: link[cur]=q
            else:
                clone=len(next); next.append(next[q].copy()); length.append(length[p]+1); link.append(0)
                while p!=-1 and next[p].get(ch)==q: next[p][ch]=clone; p=link[p]
                link[q]=link[cur]=clone
        last=cur
    print(sum(length[v]-length[link[v]] for v in range(1,len(next))))
if __name__=="__main__": main()

```

4.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

5 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.